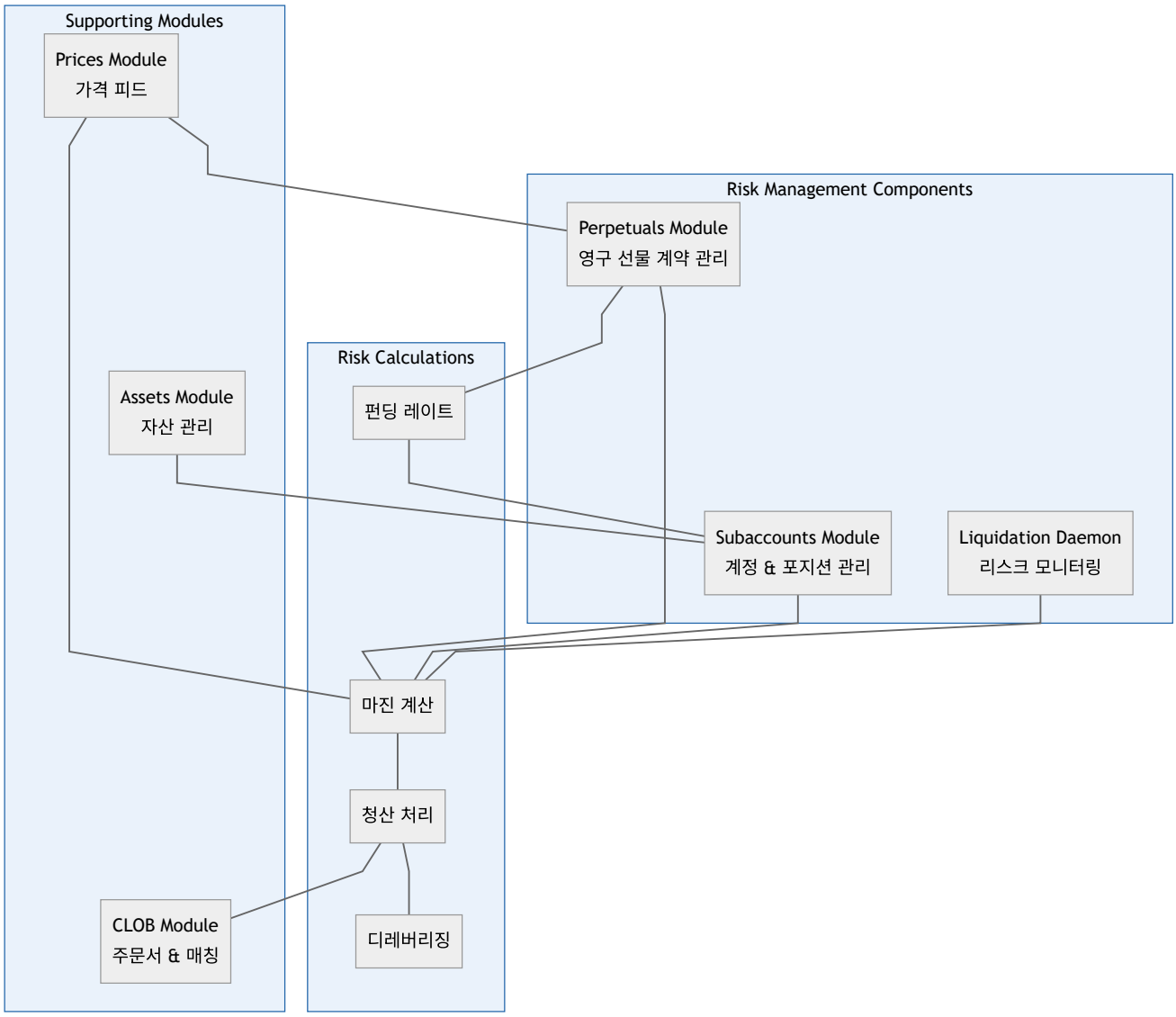
dYdX v4 Protocol Layer - 리스크 관리 및 청산 시스템 분석

개요

dYdX v4의 리스크 관리 및 청산 시스템은 영구 선물 거래소의 안정성과 건전성을 보장하는 핵심 인프라입니다. 이 시스템은 Perpetuals 모듈의 영구 선물 계약 관리, Subaccounts 모듈의 계정 및 포지션 관리, 그리고 Liquidation Daemon의 실시간 리스크 모니터링을 통해 구현됩니다. 본 문서는 이러한 컴포넌트들의 상세한 구현 방식과 상호작용을 분석합니다.

시스템 아키텍처

전체 구조



Perpetuals 모듈 - 영구 선물 계약 관리

1. 모듈 구조

위치: `protocol/x/perpetuals/`

핵심 컴포넌트:

```
type Keeper struct {
cdc codec.BinaryCodec
storeKey storetypes.StoreKey
pricesKeeper types.PricesKeeper
epochsKeeper types.EpochsKeeper
clobKeeper types.PerpetualsClobKeeper
indexerEventManager indexer_manager.IndexerEventManager
authorities map[string]struct{}
}
```

2. 영구 선물 계약 데이터 구조

Perpetual 정의

```
type Perpetual struct {
Params PerpetualParams // 계약 매개변수
FundingIndex dtypes.Int // 펀딩 인덱스
OpenInterest dtypes.Int // 미결제약정
}
```

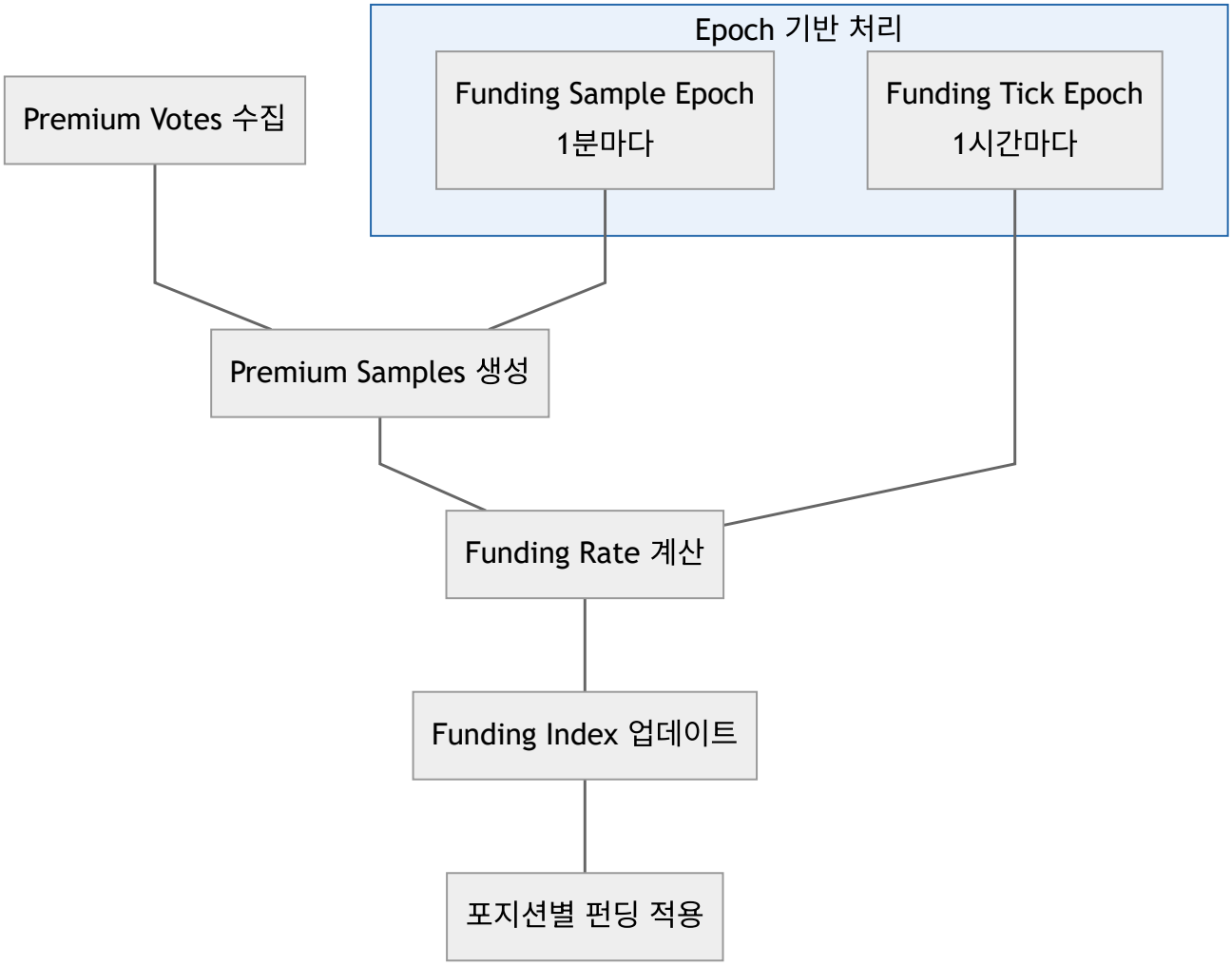
```
type PerpetualParams struct {
Id uint32 // 영구 선물 ID
Ticker string // 티커 심볼
MarketId uint32 // 시장 ID
AtomicResolution int32 // 원자 단위 해상도
DefaultFundingPpm int32 // 기본 펀딩 레이트 (ppm)
LiquidityTier uint32 // 유동성 등급
MarketType PerpetualMarketType // 시장 유형 (Cross/Isolated)
}
```

유동성 등급 (Liquidity Tier)

```
type LiquidityTier struct {
Id uint32 // 유동성 등급 ID
Name string // 등급 이름
InitialMarginPpm uint32 // 초기 마진 비율 (ppm)
MaintenanceFractionPpm uint32 // 유지 마진 비율 (ppm)
ImpactNotional uint64 // 영향 명목가치
OpenInterestLowerCap uint64 // 미결제약정 하한
OpenInterestUpperCap uint64 // 미결제약정 상한
}
```

3. 펀딩 레이트 계산 메커니즘

펀딩 레이트 산정 과정



Premium Votes 수집

```
func (k Keeper) sampleAllPerpetuals(ctx  sdk.Context) (
samples []types.FundingPremium,
err  error,
) {
allPerpetuals := k.GetAllPerpetuals(ctx)
liquidityTierToMaxAbsPremiumVotePpm := k.getLiquidityTiertoMaxAbsPremiumVotePpm(
marketIdToIndexPrice := k.pricesKeeper.GetMarketIdToValidIndexPrice(ctx)
for  _, perp := range  allPerpetuals {
```

```
indexPrice, exists := marketIdToIndexPrice[perp.Params.MarketId]
if !exists {
continue  // 유효한 인덱스 가격이 없으면 스킵
}
// 유동성 등급에 따른 영향 명목가치 가져오기
liquidityTier, err := k.GetLiquidityTier(ctx, perp.Params.LiquidityTier)
if  err != nil {
panic(err)
}
bigImpactNotionalQuoteQuantums := new(big.Int).SetUint64(liquidityTier.ImpactNot
maxAbsPremiumVotePpm := liquidityTierToMaxAbsPremiumVotePpm[perp.Params.Liquidit
// CLOB에서 프리미엄 계산
premiumPpm, err := k.clobKeeper.GetPricePremiumForPerpetual(
ctx,
perp.Params.Id,
types.GetPricePremiumParams{
IndexPrice: indexPrice,
BaseAtomicResolution: perp.Params.AtomicResolution,
QuoteAtomicResolution: lib.QuoteCurrencyAtomicResolution,
ImpactNotionalQuoteQuantums: bigImpactNotionalQuoteQuantums,
MaxAbsPremiumVotePpm: maxAbsPremiumVotePpm,
},
)
```

```
if err != nil {
return nil, err
}
if premiumPpm != 0 {
samples = append(samples, *types.NewFundingPremium(perp.Params.Id, premiumPpm))
}
}
return samples, nil
}
```

Funding Rate 계산

```
func (k Keeper) MaybeProcessNewFundingTickEpoch(ctx sdk.Context) {
// 1시간마다 실행되는 펀딩 틱 처리
numBlocks, err := k.epochsKeeper.NumBlocksSinceEpochStart(
ctx,
epochstypes.FundingTickEpochInfoName,
)
if err != nil || numBlocks != 0 {
return // 새로운 펀딩 틱 에포크가 아니면 리턴
}
allPerps := k.GetAllPerpetuals(ctx)
params := k.GetParams(ctx)
```

```
// Premium Samples에서 펀딩 레이트 계산
perpIdToPremiumPpm := k.processStoredPremiums(
ctx,
fundingTickEpochInfo,
types.PremiumSamplesKey,
minSampleRequiredForPremiumRate,
lib.AvgInt32, // 평균 계산
sampleTailsRemovalFunc, // 상하위 샘플 제거
)
for _, perp := range allPerps {
premiumPpm, found := perpIdToPremiumPpm[perp.Params.Id]
if !found {
premiumPpm = 0
}
// 펀딩 레이트 = 프리미엄 + 기본 펀딩
bigFundingRatePpm := new(big.Int).SetInt64(int64(premiumPpm))
bigFundingRatePpm.Add(
bigFundingRatePpm,
new(big.Int).SetInt64(int64(perp.Params.DefaultFundingPpm)),
)
// 유동성 등급에 따른 펀딩 레이트 제한
liquidityTier, err := k.GetLiquidityTier(ctx, perp.Params.LiquidityTier)
if err != nil {
```

```
panic(err)

}

fundingRateUpperBoundPpm := liquidityTier.GetMaxAbsFundingClampPpm(

params.FundingRateClampFactorPpm,

)

bigFundingRatePpm = lib.BigIntClamp(

bigFundingRatePpm,

new(big.Int).Neg(fundingRateUpperBoundPpm),

fundingRateUpperBoundPpm,

)

// 펀딩 인덱스 업데이트

if  bigFundingRatePpm.Sign() != 0 {

fundingIndexDelta, err := k.getFundingIndexDelta(

ctx,

perp,

bigFundingRatePpm,

fundingTickEpochInfo.Duration,

)

if  err != nil {

panic(err)

}

if  err := k.ModifyFundingIndex(ctx, perp.Params.Id, fundingIndexDelta); err !=

panic(err)
```

```
}

}

}

}
```

Funding Index Delta 계산

```
func (k Keeper) getFundingIndexDelta(

ctx  sdk.Context,

perp  types.Perpetual,

big8hrFundingRatePpm *big.Int,

timeSinceLastFunding  uint32,

) (*big.Int, error) {

marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, perp.Params.MarketId)

if  err != nil {

return  nil, fmt.Errorf("failed to get market price for perpetual %v, err = %w",

perp.Params.Id, err)

}

// 시간 비례 펀딩 레이트 계산

proratedFundingRate := new(big.Rat).SetInt(big8hrFundingRatePpm)

proratedFundingRate.Mul(

proratedFundingRate,

new(big.Rat).SetUint64(uint64(timeSinceLastFunding)),

)
```

```
proratedFundingRate.Quo(
proratedFundingRate,
new(big.Rat).SetUint64(3600*8), // 8시간 기준
)
// 펀딩 레이트를 인덱스로 변환
bigFundingIndexDelta := lib.FundingRateToIndex(
proratedFundingRate,
perp.Params.AtomicResolution,
marketPrice.Price,
marketPrice.Exponent,
)
return bigFundingIndexDelta, nil
}
```

Subaccounts 모듈 - 계정 및 포지션 관리

1. 모듈 구조

위치: `protocol/x/subaccounts/`

핵심 데이터 구조:

```
type Subaccount struct {
Id SubaccountId // 서브계정 ID
AssetPositions []AssetPosition // 자산 포지션
```

```
PerpetualPositions []PerpetualPosition  // 영구 선물 포지션
}

type SubaccountId struct {
Owner string  // 소유자 주소
Number uint32  // 서브계정 번호
}

type AssetPosition struct {
AssetId uint32  // 자산 ID
Quantums dtypes.Int  // 수량 (원자 단위)
}

type PerpetualPosition struct {
PerpetualId uint32  // 영구 선물 ID
Quantums dtypes.Int  // 포지션 크기 (양수: 롱, 음수: 숏)
FundingIndex dtypes.Int  // 펀딩 인덱스
}
```

2. 마진 계산 시스템

마진 요구사항 계산

```
func (k Keeper) GetMarginRequirements(
```

```
ctx  sdk.Context,
id  uint32,
bigQuantums *big.Int,
) (
bigInitialMarginQuoteQuantums *big.Int,
bigMaintenanceMarginQuoteQuantums *big.Int,
err  error,
) {
perpetual, marketPrice, err := k.GetPerpetualAndMarketPrice(ctx, id)
if  err != nil {
return  new(big.Int), new(big.Int), err
}
liquidityTier, err := k.GetLiquidityTier(ctx, perpetual.Params.LiquidityTier)
if  err != nil {
return  new(big.Int), new(big.Int), err
}
// 명목가치 계산
bigNotionalQuoteQuantums := GetNetNotionalInQuoteQuantums(
perpetual,
marketPrice,
new(big.Int).Abs(bigQuantums), // 절댓값 사용
)
// 초기 마진 = 명목가치 × 초기 마진 비율
```

```
bigInitialMarginQuoteQuantums = lib.BigIntMulPpm(
bigNotionalQuoteQuantums,
liquidityTier.InitialMarginPpm,
)
// 유지 마진 = 초기 마진 × 유지 마진 비율
bigMaintenanceMarginQuoteQuantums = lib.BigIntMulPpm(
bigInitialMarginQuoteQuantums,
liquidityTier.MaintenanceFractionPpm,
)
return  bigInitialMarginQuoteQuantums, bigMaintenanceMarginQuoteQuantums, nil
}
```

계정 담보 계산

```
func (k Keeper) GetNetCollateralAndMarginRequirements(
ctx  sdk.Context,
update  types.Update,
) (
risk  types.Risk,
err  error,
) {
bigNetCollateralQuoteQuantums := new(big.Int)
bigInitialMarginRequirementQuoteQuantums := new(big.Int)
```

```
bigMaintenanceMarginRequirementQuoteQuantums := new(big.Int)

// 자산 포지션에서 담보 계산

for _, assetPosition := range update.AssetUpdates {

asset, exists := k.assetsKeeper.GetAsset(ctx, assetPosition.AssetId)

if !exists {

return types.Risk{}, types.ErrAssetDoesNotExist

}

// 자산 가치를 쿼트 통화로 변환

bigAssetNotionalQuoteQuantums := k.getAssetNotionalInQuoteQuantums(

ctx,

assetPosition.AssetId,

assetPosition.BigQuantumsDelta,

)

bigNetCollateralQuoteQuantums.Add(

bigNetCollateralQuoteQuantums,

bigAssetNotionalQuoteQuantums,

)

}

// 영구 선물 포지션에서 마진 요구사항 계산

for _, perpetualPosition := range update.PerpetualUpdates {

// 포지션의 미실현 손익 계산

bigPnlQuoteQuantums, err := k.GetNetCollateral(

ctx,
```

```
perpetualPosition.PerpetualId,

perpetualPosition.BigQuantumsDelta,

)

if  err != nil {

return  types.Risk{}, err

}

bigNetCollateralQuoteQuantums.Add(

bigNetCollateralQuoteQuantums,

bigPnlQuoteQuantums,

)

// 마진 요구사항 계산

bigInitialMargin, bigMaintenanceMargin, err := k.perpetualsKeeper.GetMarginRequi

ctx,

perpetualPosition.PerpetualId,

new(big.Int).Abs(perpetualPosition.BigQuantumsDelta),

)

if  err != nil {

return  types.Risk{}, err

}

bigInitialMarginRequirementQuoteQuantums.Add(

bigInitialMarginRequirementQuoteQuantums,

bigInitialMargin,

)
```

```
bigMaintenanceMarginRequirementQuoteQuantums.Add(
bigMaintenanceMarginRequirementQuoteQuantums,
bigMaintenanceMargin,
)
}
return types.Risk{
BigNetCollateralQuoteQuantums: bigNetCollateralQuoteQuantums,
BigInitialMarginRequirementQuoteQuantums: bigInitialMarginRequirementQuoteQuantu
BigMaintenanceMarginRequirementQuoteQuantums: bigMaintenanceMarginRequirementQuo
}, nil
}
```

3. 펀딩 지불 처리

펀딩 지불 계산

```
func (k Keeper) GetSettledFunding(
ctx sdk.Context,
perpetualId uint32,
position types.PerpetualPosition,
) (*big.Int, error) {
perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, perpetualId)
if err != nil {
return nil, err
```

```
}

// 펀딩 인덱스 차이 계산

fundingIndexDelta := new(big.Int).Sub(

perpetual.FundingIndex.BigInt(),

position.FundingIndex.BigInt(),

)

// 펀딩 지불 = 포지션 크기 × 펀딩 인덱스 차이

bigFundingPaymentQuoteQuantums := new(big.Int).Mul(

position.Quantums.BigInt(),

fundingIndexDelta,

)

// 원자 단위 조정

bigFundingPaymentQuoteQuantums = lib.BigIntMulPow10(

bigFundingPaymentQuoteQuantums,

lib.QuoteCurrencyAtomicResolution,

)

return  bigFundingPaymentQuoteQuantums, nil

}
```

청산 시스템 (Liquidation System)

1. Liquidation Daemon

위치: `protocol/daemons/liquidation/`

역할: 실시간 리스크 모니터링 및 청산 대상 식별

청산 데몬 구조

```
type Client struct {
// Query clients
BlocktimeQueryClient blocktimetypes.QueryClient
SubaccountQueryClient satypes.QueryClient
PerpetualsQueryClient perptypes.QueryClient
PricesQueryClient pricestypes.QueryClient
ClobQueryClient clobtypes.QueryClient
LiquidationServiceClient api.LiquidationServiceClient
// Health monitoring
daemontypes.HealthCheckable
logger log.Logger
}
```

청산 대상 식별 프로세스

```
func (s *SubTaskRunnerImpl) RunLiquidationDaemonTaskLoop(
ctx context.Context,
client *Client,
liquidationFlags flags.LiquidationFlags,
) error {
```

```
// 1. 모든 서브계정 조회
subaccountsResponse, err := client.SubaccountQueryClient.SubaccountAll(
ctx,
&satypes.QueryAllSubaccountRequest{},
)
if err != nil {
return err
}
// 2. 현재 가격 정보 조회
marketPricesResponse, err := client.PricesQueryClient.AllMarketPrices(
ctx,
&pricestypes.QueryAllMarketPricesRequest{},
)
if err != nil {
return err
}
// 3. 영구 선물 정보 조회
perpetualsResponse, err := client.PerpetualsQueryClient.AllPerpetuals(
ctx,
&perptypes.QueryAllPerpetualsRequest{},
)
if err != nil {
return err
```

```
}
var liquidatableSubaccountIds []satypes.SubaccountId
var negativeTncSubaccountIds []satypes.SubaccountId
// 4. 각 서브계정의 담보 상태 확인
for _, subaccount := range subaccountsResponse.Subaccount {
if len(subaccount.PerpetualPositions) == 0 {
continue // 포지션이 없으면 스킵
}
// 담보 비율 계산
collateralizationRatio, err := s.calculateCollateralizationRatio(
subaccount,
marketPricesResponse.MarketPrices,
perpetualsResponse.Perpetual,
)
if err != nil {
continue
}
// 청산 조건 확인
if collateralizationRatio.Cmp(lib.OneBigRat()) < 0 {
// 유지 마진 미달 - 청산 대상
liquidatableSubaccountIds = append(liquidatableSubaccountIds, subaccount.Id)
}
// 음의 TNC (Total Net Collateral) 확인
```

```
if s.hasNegativeTotalNetCollateral(subaccount, marketPricesResponse.MarketPrice
negativeTncSubaccountIds = append(negativeTncSubaccountIds, subaccount.Id)
}
}
// 5. 청산 대상 서브계정 정보 전송
if len(liquidatableSubaccountIds) > 0 || len(negativeTncSubaccountIds) > 0 {
_, err = client.LiquidationServiceClient.LiquidateSubaccounts(
ctx,
&api.LiquidateSubaccountsRequest{
LiquidatableSubaccountIds: liquidatableSubaccountIds,
NegativeTncSubaccountIds: negativeTncSubaccountIds,
},
)
if err != nil {
return err
}
}
return nil
}
```

2. 청산 처리 프로세스

CLOB 모듈에서의 청산 처리

```go
func (k Keeper) LiquidateSubaccountsAgainstOrderbook(
ctx sdk.Context,
liquidatableSubaccountIds []satypes.SubaccountId,
) ([]types.SubaccountLiquidationInfo, error) {
var subaccountsToDeleverage []types.SubaccountLiquidationInfo
for _, subaccountId := range liquidatableSubaccountIds {
// 서브계정의 모든 포지션 조회
subaccount := k.subaccountsKeeper.GetSubaccount(ctx, subaccountId)
for _, position := range subaccount.PerpetualPositions {
if position.Quantums.IsZero() {
continue
}
// 청산 주문 생성
liquidationOrder, err := k.createLiquidationOrder(
ctx,
subaccountId,
position,
)
if err != nil {
continue
}
// 주문서에서 매칭 시도
```

```
_, orderStatus, _, err := k.MemClob.PlaceOrder(ctx, liquidationOrder)

if err != nil || orderStatus != types.Success {

// 매칭 실패 시 디레버리징 대상에 추가

subaccountsToDeleverage = append(subaccountsToDeleverage,

types.SubaccountLiquidationInfo{

SubaccountId: subaccountId,

Position: position,

})

}

}

}

return subaccountsToDeleverage, nil

}



func (k Keeper) createLiquidationOrder(

ctx sdk.Context,

subaccountId satypes.SubaccountId,

position satypes.PerpetualPosition,

) (types.Order, error) {

perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, position.PerpetualId)

if err != nil {

return types.Order{}, err

}
```

```
// 청산 주문은 현재 포지션과 반대 방향

var side types.Order_Side

if position.Quantums.Sign() > 0 {

side = types.Order_SIDE_SELL // 롱 포지션 청산

} else {

side = types.Order_SIDE_BUY // 숏 포지션 청산

}

// 청산 가격 계산 (시장가 또는 할인된 가격)

liquidationPrice, err := k.calculateLiquidationPrice(ctx, perpetual, position)

if err != nil {

return types.Order{}, err

}

return types.Order{

OrderId: types.OrderId{

SubaccountId: subaccountId,

ClientId: k.generateLiquidationClientId(),

OrderFlags: types.OrderIdFlags_Liquidation,

ClobPairId: perpetual.Params.Id,

},

Side: side,

Quantums: uint64(new(big.Int).Abs(position.Quantums.BigInt()).Uint64()),

Subticks: liquidationPrice,

TimeInForce: types.Order_TIME_IN_FORCE_IOC, // Immediate or Cancel
```

```
ReduceOnly: true,

}, nil

}
```

3. 디레버리징 (Deleveraging) 시스템

디레버리징 프로세스

```
func (k Keeper) DeleverageSubaccounts(

ctx  sdk.Context,

subaccountsToDeleverage []types.SubaccountLiquidationInfo,

) error {

for  _, liquidationInfo := range  subaccountsToDeleverage {

// 반대 포지션을 가진 계정들 찾기

counterparties, err := k.findDeleveragingCounterparties(

ctx,

liquidationInfo.Position.PerpetualId,

liquidationInfo.Position.Quantums.Sign(),

)

if  err != nil {

return  err

}

if  len(counterparties) == 0 {

// 상대방이 없으면 보험 기금 사용
```

```
err = k.useInsuranceFund(ctx, liquidationInfo)
if err != nil {
return err
}
continue
}
// 디레버리징 매칭 실행
err = k.executeDeleveraging(ctx, liquidationInfo, counterparties)
if err != nil {
return err
}
}
return nil
}


func (k Keeper) findDeleveragingCounterparties(
ctx sdk.Context,
perpetualId uint32,
positionSign int,
) ([]types.DeleveragingCounterparty, error) {
var counterparties []types.DeleveragingCounterparty
// 모든 서브계정 조회
allSubaccounts := k.subaccountsKeeper.GetAllSubaccounts(ctx)
```

```
for _, subaccount := range allSubaccounts {
for _, position := range subaccount.PerpetualPositions {
if position.PerpetualId != perpetualId {
continue
}
// 반대 방향 포지션 확인
if position.Quantums.Sign() != -positionSign {
continue
}
// 수익성 계산 (더 많은 수익을 가진 계정 우선)
pnl, err := k.calculateUnrealizedPnl(ctx, subaccount.Id, position)
if err != nil {
continue
}
if pnl.Sign() > 0 { // 수익이 있는 계정만
counterparties = append(counterparties, types.DeleveragingCounterparty{
SubaccountId: subaccount.Id,
Position: position,
Pnl: pnl,
})
}
}
}
```

```
// 수익 순으로 정렬 (높은 수익 우선)
sort.Slice(counterparties, func(i, j int) bool {
return counterparties[i].Pnl.Cmp(counterparties[j].Pnl) > 0
})
return counterparties, nil
}
```

4. 보험 기금 (Insurance Fund) 관리

보험 기금 구조

```
func (k Keeper) GetInsuranceFundName(ctx sdk.Context, perpetualId uint32) (str
perpetual, err := k.GetPerpetual(ctx, perpetualId)
if err != nil {
return "", err
}
if perpetual.Params.MarketType == types.PerpetualMarketType_PERPETUAL_MARKET_TY
// 격리 시장: 개별 보험 기금
return types.InsuranceFundName + ":" + lib.UintToString(perpetualId), nil
} else if perpetual.Params.MarketType == types.PerpetualMarketType_PERPETUAL_M
// 크로스 시장: 공통 보험 기금
return types.InsuranceFundName, nil
}
return "", fmt.Errorf("invalid market type %v for perpetual %d",
```

```
perpetual.Params.MarketType, perpetualId)
}
```

보험 기금 사용

```
func (k Keeper) useInsuranceFund(
ctx  sdk.Context,
liquidationInfo  types.SubaccountLiquidationInfo,
) error {
// 보험 기금 주소 가져오기
insuranceFundAddress, err := k.GetInsuranceFundModuleAddress(
ctx,
liquidationInfo.Position.PerpetualId,
)
if  err != nil {
return  err
}
// 보험 기금 잔고 확인
insuranceFundBalance := k.bankKeeper.GetBalance(
ctx,
insuranceFundAddress,
lib.DefaultBaseDenom,
)
```

```
// 손실 금액 계산

lossAmount, err := k.calculateLiquidationLoss(ctx, liquidationInfo)

if err != nil {

return err

}

if insuranceFundBalance.Amount.LT(lossAmount) {

// 보험 기금 부족 - 사회화 손실 처리

return k.socializeLoss(ctx, liquidationInfo, lossAmount)

}

// 보험 기금에서 손실 보전

err = k.bankKeeper.SendCoinsFromModuleToModule(

ctx,

types.InsuranceFundName,

satypes.ModuleName,

sdk.NewCoins(sdk.NewCoin(lib.DefaultBaseDenom, lossAmount)),

)

return err

}
```

리스크 매개변수 관리

1. 동적 리스크 조정

미결제약정 기반 마진 조정

```go
func (k Keeper) GetDynamicMarginRequirements(
ctx  sdk.Context,
perpetualId  uint32,
positionSize *big.Int,
) (*big.Int, *big.Int, error) {
perpetual, err := k.GetPerpetual(ctx, perpetualId)
if  err != nil {
return  nil, nil, err
}
liquidityTier, err := k.GetLiquidityTier(ctx, perpetual.Params.LiquidityTier)
if  err != nil {
return  nil, nil, err
}
// 현재 미결제약정 조회
currentOpenInterest := perpetual.OpenInterest.BigInt()
// 미결제약정에 따른 마진 조정 계수 계산
marginMultiplier := k.calculateMarginMultiplier(
currentOpenInterest,
liquidityTier.OpenInterestLowerCap,
liquidityTier.OpenInterestUpperCap,
)
// 기본 마진 요구사항 계산
```

```
baseInitialMargin, baseMaintenanceMargin, err := k.GetMarginRequirements(
ctx,
perpetualId,
positionSize,
)
if err != nil {
return nil, nil, err
}
// 동적 조정 적용
adjustedInitialMargin := new(big.Int).Mul(baseInitialMargin, marginMultiplier)
adjustedInitialMargin.Div(adjustedInitialMargin, big.NewInt(1000000)) // ppm 조정
adjustedMaintenanceMargin := new(big.Int).Mul(baseMaintenanceMargin, marginMulti
adjustedMaintenanceMargin.Div(adjustedMaintenanceMargin, big.NewInt(1000000))
return adjustedInitialMargin, adjustedMaintenanceMargin, nil
}


func (k Keeper) calculateMarginMultiplier(
currentOI *big.Int,
lowerCap uint64,
upperCap uint64,
) *big.Int {
bigLowerCap := new(big.Int).SetUint64(lowerCap)
bigUpperCap := new(big.Int).SetUint64(upperCap)
```

```
if currentOI.Cmp(bigLowerCap) <= 0 {
return big.NewInt(1000000) // 1.0 (기본 배율)
}
if currentOI.Cmp(bigUpperCap) >= 0 {
return big.NewInt(2000000) // 2.0 (최대 배율)
}
// 선형 보간
ratio := new(big.Int).Sub(currentOI, bigLowerCap)
ratio.Mul(ratio, big.NewInt(1000000))
ratio.Div(ratio, new(big.Int).Sub(bigUpperCap, bigLowerCap))
return new(big.Int).Add(big.NewInt(1000000), ratio)
}
```

2. 시장 상황별 리스크 조정

변동성 기반 조정

```
func (k Keeper) GetVolatilityAdjustedMargin(
ctx sdk.Context,
perpetualId uint32,
lookbackPeriod time.Duration,
) (*big.Int, error) {
// 과거 가격 데이터 조회
priceHistory, err := k.pricesKeeper.GetPriceHistory(
```

```
ctx,
perpetualId,
lookbackPeriod,
)
if err != nil {
return nil, err
}
// 변동성 계산 (표준편차)
volatility := k.calculateVolatility(priceHistory)
// 변동성에 따른 마진 조정
baseMarginPpm := uint32(50000) // 5% 기본 마진
volatilityAdjustment := lib.BigIntMulPpm(
big.NewInt(int64(baseMarginPpm)),
uint32(volatility*1000000), // 변동성을 ppm으로 변환
)
adjustedMarginPpm := new(big.Int).Add(
big.NewInt(int64(baseMarginPpm)),
volatilityAdjustment,
)
return adjustedMarginPpm, nil
}
```

모니터링 및 알림

1. 리스크 메트릭

시스템 리스크 지표

```
func (k Keeper) RecordRiskMetrics(ctx  sdk.Context) {
allSubaccounts := k.subaccountsKeeper.GetAllSubaccounts(ctx)
var  totalCollateral *big.Int = big.NewInt(0)
var  totalMarginRequirement *big.Int = big.NewInt(0)
var  undercollateralizedCount  int
for  _, subaccount := range  allSubaccounts {
if  len(subaccount.PerpetualPositions) == 0 {
continue
}
// 계정별 담보 및 마진 요구사항 계산
risk, err := k.subaccountsKeeper.GetNetCollateralAndMarginRequirements(
ctx,
types.Update{SubaccountId: subaccount.Id},
)
if  err != nil {
continue
}
totalCollateral.Add(totalCollateral, risk.BigNetCollateralQuoteQuantums)
totalMarginRequirement.Add(totalMarginRequirement, risk.BigMaintenanceMarginRequ
// 담보 부족 계정 수 계산
```

```
if  risk.BigNetCollateralQuoteQuantums.Cmp(risk.BigMaintenanceMarginRequirementQ

undercollateralizedCount++

}

}

// 메트릭 기록

telemetry.SetGauge(

float32(totalCollateral.Int64()),

types.ModuleName,

metrics.TotalSystemCollateral,

)

telemetry.SetGauge(

float32(totalMarginRequirement.Int64()),

types.ModuleName,

metrics.TotalMarginRequirement,

)

telemetry.SetGauge(

float32(undercollateralizedCount),

types.ModuleName,

metrics.UndercollateralizedAccounts,

)

// 시스템 담보 비율

if  totalMarginRequirement.Sign() > 0 {

collateralizationRatio := new(big.Rat).SetFrac(totalCollateral, totalMarginRequi
```

```
telemetry.SetGauge(

float32(collateralizationRatio.Float64()),

types.ModuleName,

metrics.SystemCollateralizationRatio,

)

}

}
```

결론

dYdX v4의 리스크 관리 및 청산 시스템은 영구 선물 거래소의 안정성을 보장하기 위한 다층적 보안 메커니즘을 구현합니다. Perpetuals 모듈의 정교한 펀딩 레이트 계산, Subaccounts 모듈의 실시간 마진 관리, 그리고 Liquidation Daemon의 능동적 리스크 모니터링을 통해 시장 변동성과 극단적 상황에서도 시스템의 건전성을 유지할 수 있습니다. 특히 동적 마진 조정, 다단계 청산 프로세스, 그리고 보험 기금과 디레버리징을 통한 최종 안전장치는 dYdX v4가 안전하고 신뢰할 수 있는 탈중앙화 파생상품 거래소로 운영될 수 있는 핵심 기반을 제공합니다.